Ensayo de Investigación: Persistencia de Datos

Matias Encarnación

Carrera de Computación

Universidad Nacional de Loja

10 de octubre de 2026

1. Introducción

En el presente ensayo se abordará el concepto de la persistencia de datos, analizando qué es, cuál es su utilidad práctica dentro del ecosistema del desarrollo web y por qué resulta crucial para optimizar el rendimiento y la experiencia del usuario. Asimismo, se detallarán los comandos esenciales en JavaScript para almacenar, recuperar y eliminar información localmente.

2. Desarrollo

2.1. ¿Qué hace la persistencia de datos?

La persistencia de datos en el cliente es la capacidad para conservar información localmente de forma indefinida, garantizando que los datos no se pierdan al recargar o cerrar el navegador. Mediante la API de `localStorage`, la información se guarda con clave-valor (*key-value*) asociado al dominio web correspondiente, funcionando sin fecha de expiración automática.

2.2. Utilidad e importancia

La utilidad de almacenar datos localmente radica en la reducción de solicitudes hacia el servidor y la optimización del ancho de banda. Dado que `localStorage` ofrece una capacidad aproximada de 5 MB por origen superior a los 4 KB de las *cookies*— y no envía información en las cabeceras HTTP, permite mantener estados de aplicación y otros datos de forma local, mejorando la velocidad de carga y permitiendo el funcionamiento parcial offline.

2.3. Comandos principales para la manipulación de datos

- `localStorage.setItem(clave, valor)`**:** Guarda un dato en el almacenamiento local. Recibe como parámetros una clave y un valor en (*string*); en caso de guardar objetos o arreglos, deben convertirse previamente con `JSON.stringify()`.
- `localStorage.getItem(clave)`**:** Recupera un dato previamente almacenado mediante el nombre de su clave. Devuelve el string o `null` si la clave no existe; para reconstruir estructuras complejas se emplea `JSON.parse()`.
- `localStorage.removeItem(clave)` **y** `localStorage.clear()`**:** La primera borra únicamente el elemento asociado a la clave especificada, mientras que la segunda elimina toda la caché local.

3. Ejemplos de Aplicación

1. **Preferencias de interfaz (Modo Oscuro):** Almacenar mediante `setItem('theme', 'dark')` la preferencia visual seleccionada. Al recargar el navegador, el script recupera este parámetro con `getItem('theme')` y aplica los estilos correspondientes al instante.
2. **Autoguardado de formularios y borradores:** En editores de texto o formularios extensos, cada cambio en los campos de entrada se serializa con `JSON.stringify()` y se guarda localmente. Si la página falla, los datos se restauran desde el cliente.
3. **Carrito de compras persistente en E-commerce:** Guardar el arreglo de productos seleccionados. La lista permanece disponible en su navegador, permitiéndole retomar su compra sin necesidad de iniciar sesión.

4. Conclusión

La latencia en el acceso a la información es determinante en la velocidad de respuesta: el acceso al almacenamiento local del cliente responde en microsegundos al operar directamente en la memoria del dispositivo, mientras que una consulta al servidor añade retardos de red por ida y vuelta que elevan la latencia a milisegundos. Eliminar esa dependencia de red minimiza los tiempos de renderizado e interactividad, logrando una respuesta casi instantánea que resulta indispensable en el desarrollo de aplicaciones web y otros entornos.

5. Bibliografía

Referencias

[1] M. Newage, “Persistencia de datos en JavaScript: LocalStorage,” *DEV Community*, 2022. [En línea]. Disponible en: https://dev.to/maxwellnewage/persistencia-de-datos-en-javascript-localstorage-419m.

[2] B. Semah, “Cómo usar LocalStorage en JavaScript,” *freeCodeCamp en Español*, 2024. [En línea]. Disponible en: https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/.

[3] Stack Overflow, “How to store persistent data client side,” *Stack Overflow Community*, 2022. [En línea]. Disponible en: https://stackoverflow.com/questions/714420/how-to-store-persistent-data-client-side.